Guía n°

Exchanges, bindings y routing keys aplicados al caso

ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES

Guía n°

Exchanges, bindings y routing keys aplicados al caso

Sigla	Asignatura	Experiencia de Aprendizaje
Tiempo	**Modalidad de Trabajo**	**Indicadores de Logro**

Contexto

En este tutorial, construiremos un sistema de logging realista donde los mensajes se enrutan de manera inteligente. Aprenderás a:

Usar un DirectExchange para enrutar mensajes basado en una clave.

Crear múltiples Queues (colas) para diferentes propósitos.

Utilizar múltiples Bindings con Routing Keys para enviar mensajes a colas específicas.

Implementar un backend con Spring Boot para manejar RabbitMQ.

Crear un frontend con Vite + React para interactuar con el backend.

El Escenario: Tenemos un sistema que genera logs con diferentes niveles de severidad: INFO, WARNING, y ERROR. Queremos:

Una cola que reciba todos los logs, para monitoreo general.

Una cola que reciba sólo los logs de ERROR, para un sistema de alertas críticas.

Paso 1.1: Crear el Proyecto y Dependencias

Usa Spring Initializr (start.spring.io) para crear un nuevo proyecto Maven.

Añade las siguientes dependencias:

Spring Web

Spring for RabbitMQ

Tu pom.xml debería tener esto:

```
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-amqp</artifactId>
</dependency>
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Paso 1.2: Configurar RabbitMQ (Exchanges, Queues, Bindings)

Crea una clase RabbitMQConfig.java. Aquí es donde definimos nuestra topología.

java

```
package com.example.loggingsystem;

import org.springframework.amqp.core.*;
import org.springframework.context.annotation.Bean;
```

```
import org.springframework.context.annotation.Configuration;

@Configuration
public class RabbitMQConfig {

    public static final String EXCHANGE_NAME = "logs_direct_exchange";
    public static final String ALL_LOGS_QUEUE = "all_logs_queue";
    public static final String ERRORS_ONLY_QUEUE = "errors_only_queue";

    // 1. Declarar el Exchange
    @Bean
    public DirectExchange directExchange() {
        return new DirectExchange(EXCHANGE_NAME);
    }

    // 2. Declarar las Queues
    @Bean
    public Queue allLogsQueue() {
        return new Queue(ALL_LOGS_QUEUE, true); // durable
    }

    @Bean
    public Queue errorsOnlyQueue() {
        return new Queue(ERRORS_ONLY_QUEUE, true); // durable
    }

    // 3. Declarar los Bindings
    // Cada binding une una cola a un exchange con una routing key.

    @Bean
    public Binding bindAllLogsForInfo(DirectExchange exchange, Queue
allLogsQueue) {
        return BindingBuilder.bind(allLogsQueue).to(exchange).with("INFO");
    }

    @Bean
    public Binding bindAllLogsForWarning(DirectExchange exchange, Queue
```

```
allLogsQueue) {
        return
BindingBuilder.bind(allLogsQueue).to(exchange).with("WARNING");
    }

    @Bean
    public Binding bindAllLogsForError(DirectExchange exchange, Queue
allLogsQueue) {
        return BindingBuilder.bind(allLogsQueue).to(exchange).with("ERROR");
    }

    @Bean
    public Binding bindErrorsOnly(DirectExchange exchange, Queue
errorsOnlyQueue) {
        return
BindingBuilder.bind(errorsOnlyQueue).to(exchange).with("ERROR");
    }
}
```

Explicación:

El DirectExchange reenviará los mensajes a las colas cuya routing key del binding coincida exactamente con la routing key del mensaje.

La cola all_logs_queue está enlazada 3 veces para recibir mensajes de INFO, WARNING y ERROR.

La cola errors_only_queue está enlazada 1 vez para recibir sólo mensajes de ERROR.

Paso 1.3: Crear los Consumidores

Crea una clase LogConsumer.java que escuchará ambas colas.

java

```
package com.example.loggingsystem;

import org.springframework.amqp.rabbit.annotation.RabbitListener;
import org.springframework.stereotype.Service;

@Service
public class LogConsumer {

    @RabbitListener(queues = RabbitMQConfig.ALL_LOGS_QUEUE)
    public void receiveAllLogs(String message) {
        System.out.println("[MONITOR GENERAL] Log recibido: " + message);
    }

    @RabbitListener(queues = RabbitMQConfig.ERRORS_ONLY_QUEUE)
    public void receiveErrorLogs(String message) {
        // Simular una alerta
        System.out.println("!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!");
        System.out.println("[ALERTA CRÍTICA] Error detectado: " + message);
        System.out.println("!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!");
    }
}
```

Tenemos dos métodos. Uno para cada cola. Cuando se envíe un mensaje con routing key = "ERROR", ambos métodos se ejecutarán.

Paso 1.4: Crear el Productor (REST Controller)

Este será el punto de entrada para nuestro frontend. Crea una clase LogProducerController.java.

java

```
package com.example.loggingsystem;

import org.springframework.amqp.rabbit.core.RabbitTemplate;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

@RestController
@CrossOrigin(origins = "http://localhost:5173") // Permite peticiones desde
Vite
public class LogProducerController {

    @Autowired
    private RabbitTemplate rabbitTemplate;

    @PostMapping("/log")
    public String sendLog(@RequestBody LogMessage logMessage) {
        // La routing key es el nivel del log (INFO, ERROR, etc.)
        rabbitTemplate.convertAndSend(
            RabbitMQConfig.EXCHANGE_NAME,
            logMessage.getLevel(),
            logMessage.getMessage()
        );
        return "Log enviado: " + logMessage.getMessage();
    }

    // DTO para el cuerpo de la petición
    public static class LogMessage {
        private String level;
        private String message;
        // Getters y Setters
        public String getLevel() { return level; }
        public void setLevel(String level) { this.level = level; }
        public String getMessage() { return message; }
        public void setMessage(String message) { this.message = message; }
```

```
    }
}
```

@CrossOrigin: Es crucial para permitir que el frontend (que corre en el puerto 5173) pueda hacer peticiones al backend (que corre en el 8080).

Frontend con Vite + React

Paso 2.1: Crear el Proyecto

Abre una terminal en una carpeta diferente a la de tu backend y ejecuta:

bash

```
# Sigue las instrucciones: selecciona React y luego JavaScript.
npm create vite@latest
```

Luego entra a la carpeta del proyecto e instala las dependencias:

bash

```
cd <nombre-del-proyecto>
npm install
```

Paso 2.2: Crear la Interfaz de Usuario

Reemplaza el contenido de src/App.jsx con lo siguiente:

jsx

```
import React from 'react';
import './App.css';

function App() {

 const sendLog = async (level, message) => {
   const backendUrl = 'http://localhost:8080/log';
   try {
    const response = await fetch(backendUrl, {
```

```
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
      },
      body: JSON.stringify({ level, message }),
    });
    if (!response.ok) {
      throw new Error(`HTTP error! status: ${response.status}`);
    }
    const result = await response.text();
    console.log(result);
    alert(`Log '${level}' enviado!`);
  } catch (error) {
    console.error("Error al enviar el log:", error);
    alert("Error al enviar el log. Revisa la consola.");
  }
};

 return (
  <div className="App">
    <h1>Sistema de Logging RabbitMQ</h1>
    <div className="button-container">
      <button className="info" onClick={() => sendLog('INFO', 'El usuario ha
iniciado sesión.')}>
        Enviar Log INFO
      </button>
      <button className="warning" onClick={() => sendLog('WARNING', 'El
uso de CPU está al 85%.')}>
        Enviar Log WARNING
      </button>
      <button className="error" onClick={() => sendLog('ERROR', 'No se pudo
conectar a la base de datos.')}>
        Enviar Log ERROR
      </button>
    </div>
```

```
    </div>
  );
}

export default App;
```

Y añade un poco de estilo en src/App.css:

css

```
.button-container { display: flex; gap: 1rem; justify-content: center; margin-top: 2rem; }
button { padding: 1rem; border-radius: 8px; border: none; cursor: pointer; font-size: 1rem; }
.info { background-color: #3498db; color: white; }
.warning { background-color: #f1c40f; color: white; }
.error { background-color: #e74c3c; color: white; }
```

Parte 3: Prueba Completa del Sistema

1. Asegúrate de tener RabbitMQ corriendo (por ejemplo, con Docker).
2. Inicia el Backend: Ejecuta tu aplicación de Spring Boot. Deberías ver en la consola que se conecta a RabbitMQ.
3. Inicia el Frontend: En la terminal de tu proyecto Vite, ejecuta npm run dev. Se abrirá una página web en http://localhost:5173.
4. Haz las pruebas:
    a. Haz clic en "Enviar Log INFO": En la consola de Spring Boot, verás:

    ```
    [MONITOR GENERAL] Log recibido: El usuario ha iniciado sesión.
    ```

    b. Haz clic en "Enviar Log WARNING": En la consola de Spring Boot, verás:

[MONITOR GENERAL] Log recibido: El uso de CPU está al 85%.

c. Haz clic en "Enviar Log ERROR": En la consola de Spring Boot, verás AMBOS consumidores reaccionar:

[MONITOR GENERAL] Log recibido: No se pudo conectar a la base de datos. !!!!

Links de interes

1. RabbitMQ & Spring AMQP (Backend)

Estos enlaces explican la lógica detrás de DirectExchange y BindingBuilder que usamos.

- Configuring the Broker (Documentación Oficial): Explica cómo declarar Exchanges, Queues y Bindings usando @Bean.
  - https://docs.spring.io/spring-amqp/reference/amqp/broker-configuration.html
- Sending Messages (RabbitTemplate): Detalles sobre convertAndSend y cómo funciona el enrutamiento.
  - https://docs.spring.io/spring-amqp/reference/amqp/sending-messages.html
- RabbitMQ Tutorial 4 - Routing (Java): Aunque usa la API nativa de Java, es la base conceptual de lo que implementamos (Direct Exchange).
  - https://www.rabbitmq.com/tutorials/tutorial-four-java

2. Spring Web (Comunicación Frontend-Backend)

Para entender la anotación @CrossOrigin que permitió que Vite hablara con Spring Boot.

- CORS Support in Spring Framework:
  - https://docs.spring.io/spring-framework/reference/web/webmvc-cors.html

3. Vite & React (Frontend)

Recursos oficiales para la parte del cliente que construimos.

- Vite - Getting Started:
  - https://vitejs.dev/guide/
- React - Fetch API (MDN): La documentación estándar para fetch (usado para enviar el POST).
  - https://developer.mozilla.org/es/docs/Web/API/Fetch_API/Using_Fetch